5. **An e-commerce platform has been suffering significant revenue losses due to fraudulent transactions slipping past its existing single-model detection system. The risk engineering team suspects that a model which aggregates the decisions of multiple learners would be significantly more resilient. Identify an appropriate ensemble learning technique, build it on the labelled transaction dataset, and demonstrate how it outperforms a standalone classifier.**

Random Forest is an ensemble learning algorithm that combines the predictions of multiple Decision Trees using the bagging technique and majority voting. It improves classification performance by reducing overfitting and increasing the model's robustness. In this experiment, the performance of the Random Forest classifier is compared with a standalone Decision Tree classifier for fraud detection.

A synthetic e-commerce transaction dataset (ecommerce_transactions.csv) containing 1000 labelled transaction records is used in this experiment. The dataset includes realistic transaction features such as transaction amount, customer age, purchase frequency, device risk, location risk, merchant rating, payment method, account age, login attempts, previous fraud history, and the target attribute Is_Fraud, indicating whether a transaction is genuine or fraudulent. The dataset is designed to simulate real-world e-commerce transactions while preserving data privacy.

Program Description

- Load and display the e-commerce transaction dataset.
- Split the dataset into training and testing sets.
- Train the Decision Tree and Random Forest classifiers.
- Compare their accuracy and display the confusion matrices.
- Conclude the model with better fraud detection performance.

Program

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay


df = pd.read_csv("ecommerce_transactions.csv")

X = df.drop("Is_Fraud", axis=1)
y = df["Is_Fraud"]
```

```
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.25,stratify=y,
random_state=42)

decision_tree = DecisionTreeClassifier(
    max_depth=5,
    random_state=42
)

decision_tree.fit(X_train, y_train)
dt_pred = decision_tree.predict(X_test)
dt_accuracy = accuracy_score(y_test, dt_pred)

random_forest = RandomForestClassifier( n_estimators=200,random_state=42)

random_forest.fit(X_train, y_train)
rf_pred = random_forest.predict(X_test)
rf_accuracy = accuracy_score(y_test, rf_pred)

comparison = pd.DataFrame({
    "Model": ["Decision Tree", "Random Forest"],
    "Accuracy": [round(dt_accuracy, 4), round(rf_accuracy, 4)]
})

print("\nModel Comparison\n")
print(comparison)

fig, axes = plt.subplots(1, 2, figsize=(10, 4))

ConfusionMatrixDisplay.from_predictions(
    y_test,
    dt_pred,
    cmap="Blues",
    colorbar=False,
    ax=axes[0]
)
axes[0].set_title("Decision Tree")

ConfusionMatrixDisplay.from_predictions(
    y_test,
    rf_pred,
    cmap="Blues",
    colorbar=False,
    ax=axes[1]
```

```
)
axes[1].set_title("Random Forest")

plt.tight_layout()
plt.show()

print("\nDecision Tree Accuracy :", round(dt_accuracy, 4))
print("Random Forest Accuracy :", round(rf_accuracy, 4))

if rf_accuracy > dt_accuracy:
    print("\nConclusion:")
    print("The Random Forest ensemble classifier outperformed the")
    print("standalone Decision Tree classifier.")
    print("Hence, ensemble learning is more effective for")
    print("detecting fraudulent e-commerce transactions.")
else:
    print("\nConclusion:")
    print("Both classifiers achieved similar performance.")
```